



南京航空航天大学

NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

基于大模型提示词工程的综合化航 电系统代码生成方法

南京航空航天大学/计算机科学与技术学院

汇报人 林畅 指导老师 周勇 杨志斌

2026.5.27



IMA系统架构

应用软件

导航

通信

飞控

显示

ARINC653
分区操作系统

时间分区

空间分区

通用硬件平台

开发挑战

标准复杂

手工编码

容易出错

专业门槛高

IMA系统特点

多应用集成
在统一硬件平台

严格的时间
和空间隔离

不同安全等级
应用共存

.....

流程阶段

传统流程开发

LLM辅助开发

需求分析

手动解读文档，转化为代码规范

LLM理解需求，自动识别功能点

代码编写

工程师手动编写每一行代码

LLM根据提示词自动生成代码框架

标准符合

人工检查标准符合性

提示词内嵌标准规范，生成即符合

迭代优化

手动调试修改

基于反馈自动优化提示重新生成

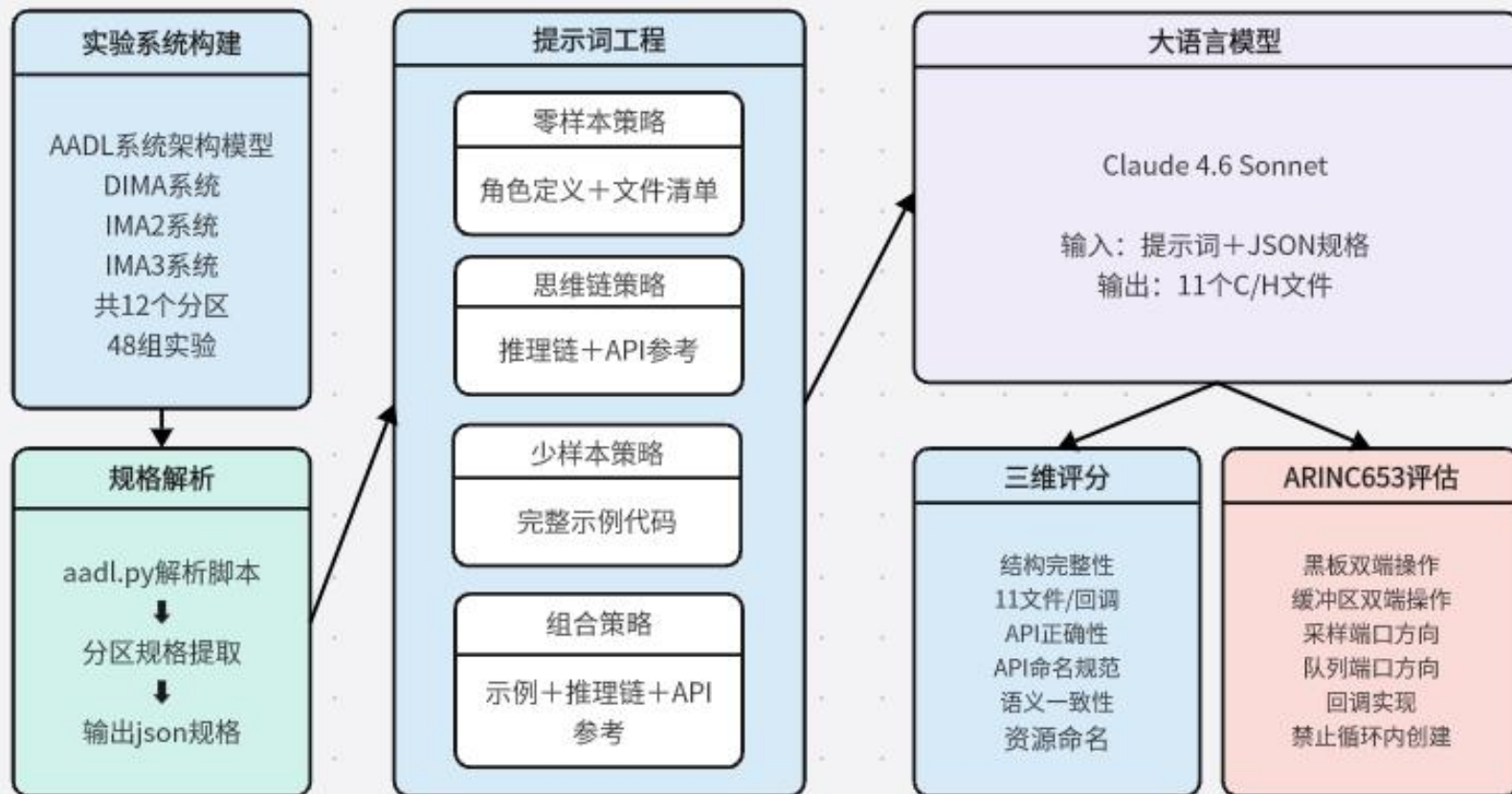
核心优势

语义理解能力

代码生成能力

零成本适配

快速迭代



输入层：AADL 系统模型 → aadl2c.py 解析 → JSON 分区规格

提示词层：4种策略（零样本 / 思维链 / 少样本 / 组合）构建提示词

Claude 4.6 Sonnet 生成 11 个 C/H 文件，三维评分 + ARINC 653 合规检查



Supported AADL → ARINC 653 mapping

```
-----
process / process implementation → partition (appMain + resource init)
thread / thread implementation → ARINC 653 PROCESS (periodic task)
out/in data port (Sampling_Refresh_Period) → SAMPLING_PORT (inter-partition)
out/in event data port (Queue_Size) → QUEUING_PORT (inter-partition)
intra-partition data port connection → BLACKBOARD (latest-value)
intra-partition event data port connection → BUFFER (FIFO queue)
data access (protected_data) → BLACKBOARD (protected shared data)
```

遍历 AADL 分区定义，提取任务、端口、黑板、缓冲区等资源，输出结构化 JSON 规格

代码片段展示了任务提取逻辑：按优先级排序分区内的线程子组件，并与线程类型定义（周期、堆栈大小）关联，生成完整的Task记录。

```
def analyse_partition(pd: PartitionDef, threads: Dict[str, ThreadDef], 1个用法
                      part_prefix: str) -> PartitionResources:
    """
    Analyse one partition definition and produce the full resource list.
    """
    pr = PartitionResources(partition=pd, prefix=part_prefix)

    # — 1. Tasks (sorted by priority for deterministic output) —————
    task_list: List[Tuple[str, str, ThreadDef]] = [] # (inst, type_base, td)
    for inst, type_base in pd.task_subcomps.items():
        td = threads.get(type_base)
        if td is None:
            print(f"[WARN] Thread type '{type_base}' not found for instance '{inst}'",
                  file=sys.stderr)
            td = ThreadDef(name=type_base, impl=type_base + '.impl')
        task_list.append((inst, type_base, td))

    task_list.sort(key=lambda t: t[2].priority)

    for idx, (inst, type_base, td) in enumerate(task_list):
        pr.tasks.append(TaskRes(inst_name=inst, type_name=type_base,
                                thread=td, index=idx))
```



零样本提示

1 角色定义

"你是一名嵌入式航空软件工程师，熟悉 ARINC 653 标准和 ACoreOS653 实时操作系统"

2 分区规格

由上一步的JSON解析获得

3 具体文件要求

对每个文件进行约束，同时约束命名规范等细节

你是一名嵌入式航空软件工程师，熟悉 ARINC 653 标准和 ACoreOS653 实时操作系统。

请根据以下 IMA（综合模块化航空电子）分区规格，生成完整的 ARINC 653 C 代码。该分区运行在 ACoreOS653 操作系统上，使用 APEX 服务接口。

分区规格

```
{
  "partition": "ps2",
  "module": "M1",
  "tasks": [
    {"name": "task21", "period_ms": 50, "priority": 2},
    {"name": "task22", "period_ms": 50, "priority": 3},
    {"name": "task23", "period_ms": 100, "priority": 4}
  ],
}
```

要求

请生成以下 11 个文件的完整代码：

1. **deployment.h** - 使用 `#define` 定义部署常量 (NB_THREADS、NB_SAMPLINGS、NB_QUEUEINGS、NB_BLACKBI
2. **deployment.c** - 仅包含 `#include "deployment.h"`
3. **gtypes.h** - 定义 `typedef int integer;` (映射 AADL Base_Types::Integer)
4. **gtypes.c** - 仅包含 `#include "gtypes.h"`
5. **globals.h** - 定义 `CHECK\_CODE(msg, code)` 宏: NO_ERROR 时打印成功，否则打印真实错误码
6. **globals.c** - 仅包含 `#include "globals.h"`
7. **subprograms.h** - 声明规格中列出的子程序函数 (若无则为空头文件)
8. **subprograms.c** - 子程序桩实现 (函数体为 TODO 注释)
9. **activity.h** - 声明所有 `taskXX\_job(void)` 函数
10. **activity.c** - 实现所有任务函数，每个任务有 `while(1)` 循环，末尾调用 `PERIODIC\_WAIT`
11. **main.c** - 实现 `appMain(void)`：依次 CREATE 所有端口/黑板/缓冲区/进程，最后调用 `SET\_PARTITION`



思维链提示

1 以零样本提示词为基础
包含零样本提示词的三个部分

2 添加API参考模块
APEX API是航空私有接口，训练数据中很少出现，模型容易产生幻觉

3 四步推理引导
(1) 分析端口和通信资源
(2) 规划appMain的CREATE顺序
(3) 分析每个任务的通信行为
(4) 最后才生成代码
迫使模型先做全局规划，避免端口遗漏问题

你是一名嵌入式航空软件工程师，熟悉 ARINC 653 标准和 ACoreOS653 实时操作系统。

请根据以下分区规格，先逐步分析，再生成完整的 ARINC 653 C 代码。

分区规格

```
{{SPEC_JSON}}
```

ARINC 653 APEX API 参考

端口创建 (在 appMain 中调用一次)

```
/* 采样端口 */
CREATE_SAMPLING_PORT(
    "端口名", /* PORT_NAME */
    sizeof(integer), /* MAX_MESSAGE_SIZE */
    SOURCE或DESTINATION, /* PORT_DIRECTION */
    刷新周期_ns, /* REFRESH_PERIOD (纳秒) */
    &id变量, /* PORT_ID */
    &ret);
```

请按以下步骤推理，然后生成代码

步骤 1: 分析端口和通信资源

- 列出所有需要创建的端口 (类型、名称、方向)
- 列出所有黑板和缓冲区

步骤 2: 规划 appMain 的 CREATE 顺序

- 通常顺序: 先创建通信资源, 在其依赖任务之前; 最后 SET_PARTITION_MODE(NORMAL)
- 列出 CREATE 调用序列

步骤 3: 分析每个任务的通信行为

- 对每个任务: 它读哪些资源? 写哪些资源? 调用哪些子程序?
- 推断 activity.c 中的 READ/WRITE 操作序列
- 命名确认: 列出每个任务的函数名 (必须为 `tasks[].name + _job`, 如 `task31_job`)

步骤 4: 生成 11 个文件的完整代码

请先完成步骤 1-3 的分析, 再输出步骤 4 的代码。

代码规范

- 任务函数命名 (严格): 每个任务函数名必须为 `<name>_job`, 其中 `<name>` 完全等于规格 JSON 中 `tasks[].name` 的值。例如 `"name": "task31"` → 函数签名为 `void *task31_job(void *arg)`。activity.h 声明、activity.c 实现、`tattr.ENTRY_POINT` 赋值、`strcpy(tattr.NAME, ...)` 字符串, 均必须使用完全相同的名称。



少样本提示

在零样本提示词的基础上，添加一个完整分区的11文件示例代码

组合提示

在零样本提示词的基础上，添加完整示例代码和API参考
同时引导模型分步推理

示例输出：分区 ps3 的 11 个 C 文件

deployment.h

```
#ifndef __PS3_GENERATED_DEPLOYMENT_H_
#define __PS3_GENERATED_DEPLOYMENT_H_

#define IMA2C_RUNTIME_ACoreOS653 1
#define ACoreOS653_GENERATED_CODE 1
#define ACoreOS653_CONFIG_NB_THREADS 3
#define ACoreOS653_CONFIG_NB_SAMPLINGS 0
#define ACoreOS653_CONFIG_NB_QUEUEINGS 2
#define ACoreOS653_CONFIG_NB_BLACKboards 0
#define ACoreOS653_CONFIG_NB_BUFFERS 0
#define ACoreOS653_NEEDS_ARINC653_PARTITION 1
#define ACoreOS653_NEEDS_ARINC653_PROCESS 1
#define ACoreOS653_NEEDS_ARINC653_SAMPLING 0
#define ACoreOS653_NEEDS_ARINC653_QUEUEING 1
#define ACoreOS653_NEEDS_ARINC653_BLACKBOARD 0
#define ACoreOS653_NEEDS_ARINC653_BUFFER 0
#define ACoreOS653_NEEDS_ARINC653_SEMAPHORE 0
#define ACoreOS653_NEEDS_ARINC653_EVENT 0
#define ACoreOS653_NEEDS_MIDDLEWARE 1
#define ACoreOS653_NEEDS_ARINC653_TIME 1
#define ACoreOS653_CONFIG_STACKS_SIZE 24576

#endif
```



三维检查

结构
完整性

11个文件是否齐全,
HM回调是否存在、
函数签名、头文件保
护、宏定义

API正确性

CREATE/READ/WRITE
配对, 端口方向, 命名
格式

语义
一致性

任务名、端口名、
资源名与 JSON
规格一致

ARINC 653 合规检查

R1/R2: 黑板/缓冲区必须
同时有读写两端操作

R3-R6: 采样/队列端口方向
与规格一致

R7: 必须实现 HM 回调
函数

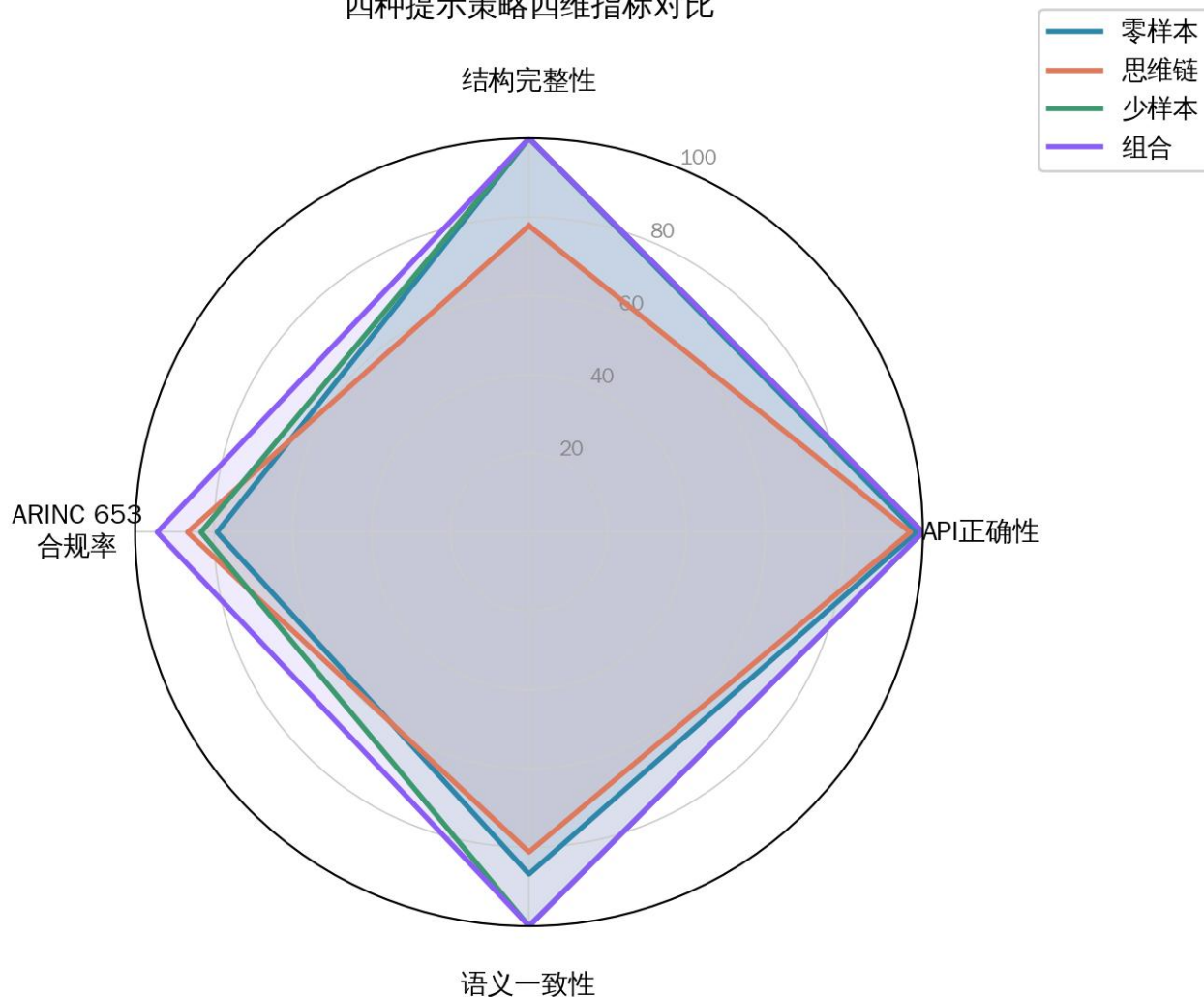
R8: CREATE_* 不得出现
在 while(1) 循环内

metrics

- `_init_.py`
- `api_check.py`
- `arinc_compliance.py`
- `semantic.py`
- `structural.py`

```
structural.py x api_check.py
1  """
2  维度1: 结构完整性 (Structural Completeness) 权重 30%
3
4  检查项:
5  S1 必需文件是否全部存在 (11个文件)
6  S2 main.c 中是否定义了 appMain()
7  S3 activity.h 中是否声明了所有 taskXX_job
8  S4 activity.c 中是否实现了所有 taskXX_job
9  S5 所有 .h 文件是否有头文件保护 (#ifndef ... #define ... #endif)
10 S6 所有 .c 文件是否包含必要的 #include (apexLib.h)
11 S7 globals.h 中是否定义了 CHECK_CODE 宏
12 S8 gtypes.h 中是否定义了 integer 类型
13 """
14
15 > import ...
16
17
18
19
20 > def _read(path: str) -> str:...
21
22
23
24
25
26
27
28
29 > def check(gen_dir: str, spec: dict) -> Dict:...
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
```


四种提示策略四维指标对比



零样本和思维链实验以5分区的DIMA系统为生成目标
少样本和组合实验以DIMA的一个分区为示例，生成3分区的IMA2系统代码

少样本和组合策略三维得分均达满分；
组合策略合规率最高（93.5%），是最佳策略

CoT 推理自由度过高，自主调整了文件职责和命名风格，偏离了评估规则期望的固定格式，体现了提示词约束不足时 LLM 输出的不稳定性。



常见错误分析

策略/分区	三维综合得分	ARINC653 合规率	失分原因
零样本/ps1	94.4	66.7%	C6/C7 通过（名称出现在 extern 中），但 R1/R2 失败（从未实际调用 API）
少样本/PB	100.0	83.3%	C6 通过（黑板名出现），但 R1 失败（缺少 READ_BLACKBOARD 调用）
思维链/ps1	81.4	50.0%	A10/A11 失分与 R3/R6 失分来自同一根本问题（端口 API 方向错误）

复杂系统泛化验证

前述实验已系统验证了四种策略在两套 IMA 系统上的表现
以复杂系统 IMA3 的 4 个分区来验证少样本和组合策略的泛化能力

即使目标系统的复杂分区的资源密度大幅提升，策略性能仍然保持稳定

系统	提示策略	三维均值	合规率均值
IMA2	少样本	100.0	83.3%
IMA2	组合	100.0	94.4%
IMA3	少样本	100.0	85.1%
IMA3	组合	100.0	92.9%



本分区名称为 PA，属于 IMA 系统 MA 模块。包含 1 个周期性任务，任务名称为 taskA1，周期为 50 毫秒，优先级为 2，栈大小 8192 字节。分区有 1 个队列通信端口，名称为 ctrl_in，方向为接收（从外部接收消息），最大队列深度为 4 条消息，排队策略为先进先出。该分区没有采样端口、没有黑板资源、没有缓冲区资源、没有子程序。

本分区名称为 PB，属于 IMA 系统 MA 模块。包含 2 个周期性任务：taskB1，周期 25 毫秒，优先级 2，栈大小 8192 字节；taskB2，周期 50 毫秒，优先级 3，栈大小 8192 字节。分区有以下通信资源：1 个采样端口 sensor_out，方向为发送（向外部写出数据），刷新周期 100 纳秒；1 个队列端口 cmd_in，方向为接收，最大队列深度 4，先进先出；1 个黑板资源，名称为 bb_data_link，用于分区内任务间数据共享（taskB1 写入，taskB2 读取）。该分区没有缓冲区资源、没有子程序。

线程定义包 (DIMA2_threads)

```
package DIMA2_threads
public

  with ARINC653;
  with Base_Types;

  thread TskA1
    features
      ctrl_in: in event data port Base_Types::Integer {Queue_Size => 4;
        ARINC653::Timeout => 5ms;
        ARINC653::Queueing_Discipline => FIFO;};
    end TskA1;

  thread implementation TskA1.impl
    properties
      Dispatch_Protocol    => Periodic;
      Period                => 50ms;
      Deadline              => 50ms;
      Priority              => 2;
      Stack_Size            => 8Kbyte;
      ARINC653::Time_Capacity => 50ms;
      ARINC653::Deadline_Type => soft;
    end TskA1.impl;
```

输入格式	综合得分	合规率	问题
自然语言描述	100.0	66.7%	黑板/缓冲区操作缺失
AADL 原文	91.2	100.0%	资源命名偏差
结构化 JSON	97.2	66.7%	黑板/缓冲区操作缺失

自然语言综合得分最高，LLM 对自然语言理解更好
AADL 原文的653合规率最高 其原文中直接包含了部分 API ；
JSON 格式在两者间取得平衡，且更易于自动化生成



Supported AADL → ARINC 653 mapping

```
process / process implementation → partition (appMain + resource init)
thread / thread implementation → ARINC 653 PROCESS (periodic task)
out/in data port (Sampling_Refresh_Period) → SAMPLING_PORT (inter-partition)
out/in event data port (Queue_Size) → QUEUING_PORT (inter-partition)
intra-partition data port connection → BLACKBOARD (latest-value)
intra-partition event data port connection → BUFFER (FIFO queue)
data access (protected_data) → BLACKBOARD (protected shared data)
```

从 AADL 提取 JSON 规格

你是一名嵌入式系统软件工程师，熟悉 ARINC 653 标准，能够根据 AADL 模型生成符合 ARINC 653 规范的代码。请根据以下 AADL 模型生成符合 ARINC 653 规范的代码。生成代码时，请确保代码符合 ARINC 653 规范，并生成符合 ARINC 653 规范的代码。

PASTED

我将根据规格生成所有11个文件的完整代码。

```
=== deployment.h ===

c

#ifndef DEPLOYMENT_H
#define DEPLOYMENT_H

#define ACOREOS653_CONFIG_NB_THREADS 3
#define ACOREOS653_CONFIG_NB_SAMPLINGS 1
#define ACOREOS653_CONFIG_NB_QUEUEINGS 1
#define ACOREOS653_CONFIG_NB_BLACKBOARDS 1
#define ACOREOS653_CONFIG_NB_BUFFERS 5
#define ACOREOS653_NEEDS_STACKS_SIZE 24576

#endif /* DEPLOYMENT_H */
```

将 嵌入JSON的提示词模板发给 Claude，得到 11 个文件

IMA 代码评估工具

单分区评估 | ARINC 653 合规检查 | 编译检查解析

参数设置

生成代码目录
C:/Users/86138/Desktop/毕设代码/g 浏览

分区
pa

实验标签 (可选)

☐ 保存结果 (JSON)

开始评估

评估完成 综合得分 51.1 分

评分结果

维度1 结构完整性 权重 30%
维度2 API 正确性 权重 40%
维度3 语义一致性 权重 30%
综合得分 加权平均

综合评级: 一般 — 基本框架正确，API / 语义有较多缺失

检查明细

项目	检查内容	状态	得分	备注
—— 维度1 结构完整性 ——				
S1	必需文件完整性 (11个文件)	✓	100%	全部存在
S2	main.c 中定义了 appMain(void)	✓	100%	
S3	activity.h 声明了所有 1 个任务函数	✗	0%	缺失声明: [taskA1]
S4	activity.c 实现了所有 1 个任务函数	✗	0%	缺失实现: [taskA1]
S5	所有 .h 文件有头文件保护 (#ifndef/#	✓	100%	全部正确
S6	activity.c 和 main.c 包含 apexLib.h	✓	100%	全部包含
S7	globals.h 定义了 CHECK_CODE 宏	✓	100%	正确
S8	globals.h 中 typedef int integer	✓	100%	
—— 维度2 API 正确性 ——				
A1	CREATE_SAMPLING_PORT 次数 (期望 0 次)	△	50%	期望0次, 实际1次
A2	CREATE_QUEUEING_PORT 次数 (期望 1 次)	✓	100%	期望1次, 实际1次
A3	CREATE_BLACKBOARD 次数 (期望 0 次)	✗	0%	期望0次, 实际5次
A4	CREATE_BUFFER 次数 (期望 0 次)	✗	0%	期望0次, 实际3次
A5	CREATE_PROCESS 次数 (期望 1 次)	✗	0%	期望1次, 实际4次
A6	START 次数 (期望 1 次)	✗	0%	期望1次, 实际4次
A7	main.c 末尾调用 SET_PARTITION_M	✓	100%	
A8	每个任务函数体包含 PERIODIC_WAIT	✗	0%	缺少PERIODIC_WAIT: [taskA1]
A9	CHECK_CODE 调用次数合理 (期望 ≥ 25 次)	✓	100%	实际25次, 最少期望4次

IMA 代码评估工具 v1.0 — DIMA/eval/gui.py

进行三维评估及ARINC653评估



名称

- .gitkeep
- activity.c
- activity.h
- deployment.c
- deployment.h
- globals.c
- globals.h
- gtypes.c
- gtypes.h
- main.c
- subprograms.c
- subprograms.h

每组实验均生成11个文件

```
/*=====
 * appMain
 * 分区: ps2, 模块: M1, AADL进程: DIMA_partitions::P2.impl
 * 创建顺序: 端口 → 黑板 → 缓冲区 → 任务 → SET_PARTITION_MODE
 *=====
void appMain(void)
{
    RETURN_CODE_TYPE    ret;
    PROCESS_ATTRIBUTE_TYPE tattr;

    /*-----
     * 1. 创建采样端口
     *   pr2samplingin: DESTINATION, 刷新周期 50ms
     *-----
    CREATE_SAMPLING_PORT(
        "pr2samplingin",
        (MESSAGE_SIZE_TYPE)sizeof(integer),
        DESTINATION,
        PR2SAMPLINGIN_REFRESH_PERIOD,
        &ps2_pr2samplingin_id,
        &ret);
    CHECK_CODE("CREATE_SAMPLING_PORT pr2samplingin", ret);
}
```

资源名与 JSON 规格完全一致

```
/*-----
 * 5a. 创建并启动 task21 (50ms, 优先级 2)
 *-----
memset(&tattr, 0, sizeof(tattr));
strcpy(tattr.NAME, "task21");
tattr.ENTRY_POINT = task21_job;
tattr.BASE_PRIORITY = 2;
tattr.PERIOD = 50000000ll;
tattr.STACK_SIZE = TASK_STACK_SIZE;
tattr.TIME_CAPACITY = 50000000ll;
tattr.DEADLINE = SOFT;
CREATE_PROCESS(&tattr, &arinc_threads[0], &ret);
CHECK_CODE("CREATE_PROCESS task21", ret);
START(arinc_threads[0], &ret);
CHECK_CODE("START task21", ret);
```

周期正确换算为纳秒

```
/*-----
 * 6. 切换分区模式为 NORMAL
 *-----
SET_PARTITION_MODE(NORMAL, &ret);
CHECK_CODE("SET_PARTITION_MODE NORMAL", ret);
```

SET_PARTITION_MODE 在最后调用



主要贡献

工具链: aadl2c.py 实现
AADL → JSON 规格自动
提取

对比实验: 系统比较四种提
示词策略在 IMA 代码生成中
的表现

评估体系: 三维评分+
ARINC 653 合规检查

结论

少样本/组合策略综合得分
达 100分, 合规率最高
93.5%

LLM 能够生成质量接近手工
编写的 ARINC 653 分区代码

CoT 策略存在"过度生成"
问题, 需要严格约束输出
格式

未来展望

扩展到更多 IMA 系统和
RTOS 平台

引入运行时仿真验证

探索 Fine-tuning 替代提
示词工程



— — —
请老师批评指正